Smart-X — Project Completion Report

Module: PROG7312
Project: Smart-X Simulated IoT Gateway
Scope: Part 1 (Telemetry) · Part 2 (Command Stream) · Final PoE Part 3 (Network Topology & Mesh Routing)
Repository: https://github.com/akzz98/SmartX (branch `POE`)
Stack: ASP.NET Core API + Blazor WebAssembly + SmartX.Shared (.NET 10)

---

1. Executive summary

Smart-X is a simulated South African IoT gateway for hydroponics, utilities, and site-meter telemetry. Heterogeneous ESP32 packets (environmental floats, power integers, actuator booleans) are validated and stored by an ASP.NET Core API and investigated in a Blazor WebAssembly operator console.

The Final PoE delivers a **unified three-pillar application**:

Pillar	Focus	Operator entry
Part 1	Sensor registration, typed ingest, Progressive Disclosure, health/freshness	`/telemetry`
Part 2	Real-time command stream: queues, undo stack, timeline, HashSet alerts, suggestions	`/commands`
Part 3	AVL history trees, mesh graph, BFS reachability, Prim MST, topology dashboard	`/mesh`

One `GatewayStore` singleton and one boot sequence seed the fleet, faults, queues, and mesh so a single API restart demos the full arc.

---

2. Project arc — Part 1 to Final PoE

2.1 Part 1 — Available Sensor Data Ingestion and Telemetry

Goal. Ingest heterogeneous telemetry and let operators investigate the fleet through Progressive Disclosure (overview → filters → device detail → history) without hiding silent or faulty devices.

Delivered:

- Nested deployment forest (site → zone → sub-zone → node) with recursive validation
- Registration and typed ingest for environmental, power, and actuator packets
- Health and freshness classifiers (live / aging / stale / disconnected)
- Seeded fault scenarios (temperature spike, stuck actuator, silent sensor, malformed frames, location outage, recovery)
- Multipart diagnostic attachments

- Blazor L0–L3 investigation UI

Structures emphasised: lists, dictionaries for O(1) device resolve, recursive tree validation for deployment.

2.2 Part 2 — Real-Time Command Stream and History

Goal. Process critical alerts ahead of FIFO backlog, support manual actuator overrides with undo, track unique disconnect/error keys, and surface predictive suggestions.

Delivered:

- Standard FIFO ingest queue vs critical priority bypass queue
- Background `TelemetryQueueConsumer` (hosted service) — enqueue-only on request path
- Priority severity ranking with **RaisedAt** tie-breaker
- Command undo stack and SortedDictionary timeline
- HashSet unique alert keys (disconnect / error) with duplicate suppression
- Recommendation engine with normalised confidence scores
- `/commands` console with filters, issue/undo, suggestions, and 5s auto-refresh

Lecturer feedback (Part 2) drove Final PoE Stage 1 fixes: session state across navigation, pillar-grouped menu, auto-refresh, background consumer, tie-breaker, disconnect detection, and confidence-based suggestions (see `docs/FEEDBACK-UPDATES.md`).

2.3 Part 3 — Network Topology and Mesh Routing

Goal. Model the radio mesh, query historical telemetry efficiently, traverse for reachability/healing, and compute a power-efficient routing backbone to the gateway.

Delivered:

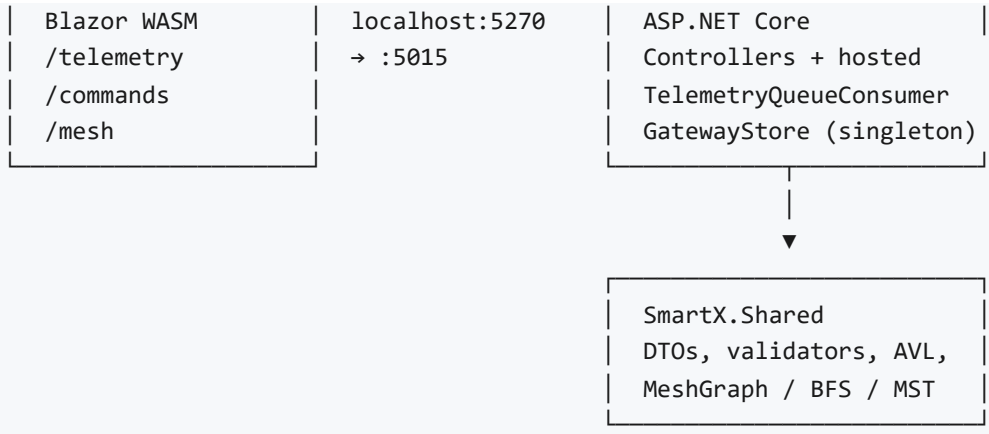
Structure / algorithm	Role	Evidence
AVL trees (per device)	Timestamp range history without full list scan (~O(log n + k))	`AvlTelemetryHistoryTree`, `GET …/history/range`, `/mesh` historical log
Undirected mesh graph	Gateway + 16 ESP32s, ≥20 weighted wireless links	`MeshGraph`, `MeshTopologySeeder`, `GET /api/mesh/topology`
BFS / DFS	Live reachability; down vs cut-off; alternate hops	`MeshGraphTraverser`, `MeshReachabilityAnalyzer`, `GET /api/mesh/traversal`
Prim’s MST	Minimum-cost backbone; parent hops toward gateway	`MeshMstBuilder`, `GET /api/mesh/mst` (~25→16 links, ~47% cost saved)
Topology UI	SVG mesh, route track by id/MAC, operator briefing, AVL log	`/mesh`

Seeded mesh fault: `MeshDownScenarioSeeder` forces `sx-pwr-inverter` offline so healing hops are visible alongside the silent sump disconnect from Part 1/2.

---

3. Architecture of the unified system

```
┌──────────────────────┐      HTTP/JSON      ┌──────────────────────────────┐
│  SmartX.Client       │ ──────────────────► │  SmartX.Api                  │
```




```

- **CORS:** browser origin `http://localhost:5270` → API `http://localhost:5015`
- **Optional Docker:** `docker compose up --build` maps the same host ports (see README)
- **No database:** in-memory store; restart API to reset seeds

---

## 4. How to demonstrate the completed system

**Recorded demo (YouTube):** [https://youtu.be/33HNgywCXoc](https://youtu.be/33HNgywCXoc)

Walkthrough for live marking or self-check:

1. Start API then client (`dotnet run` **or** `docker compose up --build`).
2. Open `http://localhost:5270` — all three pillars **Available**.
3. **Part 1:** `/telemetry` — 16 devices, exceptions, open a critical device.
4. **Part 2:** `/commands` — FIFO peak, suggestions, unique keys, issue/undo override.
5. **Part 3:** `/mesh` — red down nodes, track `sx-env-rack1-moist` MST path, operator briefing (mesh vs MST savings), AVL historical log.

API smoke:

```bash
curl http://localhost:5015/api/sensors/summary
curl http://localhost:5015/api/stream/snapshot
curl http://localhost:5015/api/mesh/topology
curl http://localhost:5015/api/mesh/traversal
curl http://localhost:5015/api/mesh/mst
```

---

5. Feedback-driven improvements (Parts 1–2 → Final PoE)

Part 2 critique items were closed before/during Part 3 work and logged in `docs/FEEDBACK-UPDATES.md`, including:

- Operator session state surviving navigation
- Pillar-grouped navigation (not a flat page list)
- Command-stream auto-refresh
```

- Background queue consumer (no sync drain on ingest)
- Priority queue time tie-breaker
- Freshness-based disconnect detection
- Confidence-scored suggestions without a hard-coded moisture rule

These remain active in the unified app and support the Final PoE “full functioning” requirement.

---

6. Task 3 — architectural and algorithmic challenges (and how they were overcome)

Final PoE Task 3 (Network Topology and Mesh Routing) was not a greenfield microservice. It had to land **inside** the existing Part 1/2 gateway without breaking Progressive Disclosure or the command stream. The challenges below are specific to this codebase.

6.1 Shared contracts across API and Blazor WASM

Challenge. The Blazor client must display topology, traversal, MST, and AVL range results without re-implementing graph logic in JavaScript. Enums (`FreshnessState`, `MeshNodeKind`, …) and nested DTOs break easily if JSON shape drifts.

Overcome.

- All mesh/AVL models and DTOs live in **`SmartX.Shared`**, referenced by both Api and Client.
- API and client agree on **camelCase** property names and **string enums** (`JsonStringEnumConverter`).
- Client uses a single `HttpClient` with `ApiBaseUrl` (`wwwroot/appsettings.json`) and `GatewayJson.Options` for deserialize.
- CORS explicitly allows `http://localhost:5270` → `http://localhost:5015` (and Docker’s `appsettings.Docker.json`).

Evidence: `MeshController` returns `MeshTopologySnapshotResponse` / `MeshTraversalResponse` / `MeshMstResponse`; `/mesh` binds the same types.

6.2 Choosing and integrating an AVL history tree (not a list scan)

Challenge. Part 1 already stored packets in lists for Progressive Disclosure history. Task 3 required **range queries** (e.g. moisture for a device between two timestamps) that stay efficient as history grows — a full list scan would fail the “trees for historical queries” criterion and look identical to Part 1.

Overcome.

- Introduced a **per-device AVL** (`AvlTelemetryHistoryTree`) keyed by timestamp then sequence, indexed via `TelemetryHistoryIndex`.
- Every successful ingest also inserts into the tree (same write path as list history).
- `QueryRange` prunes subtrees outside `[from, to]` and returns **`nodesVisited`** vs **`treeSize`** so the UI can prove ≈ O(log n + k) behaviour.
- `/mesh` historical log and `GET /api/sensors/{id}/history/range` share that path.

Evidence: Demo device `sx-env-rack1-moist` with metric `Moisture`; range payload shows match count and visit count.

6.3 Modelling a realistic mesh on top of an existing fleet

Challenge. The graph could not be a toy triangle graph. It had to use the **seeded 16 ESP32s**, include a gateway vertex, multi-neighbour links, and weights that mean something for MST (power/RSSI-style cost) — while staying consistent with live freshness (online/offline).

Overcome.

- `MeshGraph` adjacency list: nodes = gateway + sensors; undirected weighted edges.
- `MeshTopologySeeder` builds ≥20 links (rack clusters, feeder bay, inter-rack backbone).
- `RefreshOnlineStatus` reclassifies from `LastSeenAt` so Part 1/2 disconnects (e.g. silent sump) appear as mesh-down.
- `MeshDownScenarioSeeder` ages `sx-pwr-inverter` so BFS healing is always demoable after boot.

Evidence: `GET /api/mesh/topology` → 17 nodes / 25 edges; offline count includes inverter + sump.

6.4 Traversal semantics: topology vs live routing

Challenge. A single BFS over all edges cannot answer both “ideal path if everything were up” and “who can still forward now.” Operators also need **alternate hops** when a relay is down — not only a visited-order list.

Overcome.

- `MeshGraphTraverser` supports BFS and DFS with an **`onlineOnly`** flag (skip offline neighbours).
- `MeshReachabilityAnalyzer` runs topology BFS + live BFS, then classifies **reachable / down / cut-off**.
- For each down node it suggests a live neighbour closest (by hop) to the gateway and exposes ideal vs alternate paths.
- API: `GET /api/mesh/traversal`; UI: operator briefing in plain language on `/mesh`.

Evidence: Traversal lists `sx-pwr-inverter` and `sx-env-sump-temp` as down with suggested next hops (typically `sx-gateway`).

6.5 MST choice (Prim) and comparing it to the full mesh

Challenge. Rubric accepts Prim or Kruskal, but the product need is a **gateway-rooted routing tree** with parent hops for telemetry, plus a clear “full mesh vs efficient backbone” story for marks and demos.

Overcome.

- Implemented **Prim** in `MeshMstBuilder` (documented why not Kruskal: root growth, adjacency-list fit, natural parents).
- `onlineOnly=true` MST drives route tracking on `/mesh`; full MST payload includes **`MeshVsMstComparison`** (edges saved, cost saved, redundant link list).
- Seeded numbers are stable for demos: ~25 links / cost 42.7 → 16 links / cost 22.7 (~47% radio cost saved).

Evidence: `GET /api/mesh/mst` comparison block; SVG path highlight for `sx-env-rack1-moist → … → sx-gateway`.

6.6 Keeping Part 1 and Part 2 alive under Part 3 load

Challenge. Adding mesh APIs and a heavy `/mesh` page risked regressions: queue consumer starvation, blocking UI, or treating Part 3 as a separate app (against the brief).

Overcome.

- Mesh reads refresh status under the existing `GatewayStore` lock; they do not replace ingest/queue paths.
- Hosted `TelemetryQueueConsumer` continues priority-then-FIFO drain independently of topology GETs.
- `/mesh` loads topology, MST, traversal, and history asynchronously with explicit API-down / empty / error callouts (no silent hang).
- Startup menu and sidebar keep three pillars; README documents one-boot demos across all three.

Evidence: Fresh-restart smoke (checklist Stage 9.3): summary, stream snapshot/suggestions, mesh topology/traversal/MST all green together.

---

7. Key learnings (linked to actual Smart-X tasks)

These lessons come from implementing Parts 1–3 in one gateway — not from generic textbook summaries.

7.1 Data structures — pick the structure that matches the operator question

Operator question	Structure used in Smart-X	Why it fits
“Which device is this id/MAC?”	Dictionaries in `SensorFleet`	O(1) resolve on every ingest and command
“What happened to overrides/alerts over time?”	`SortedDictionary` timeline	Ordered history without re-sorting a list each GET
“Is this disconnect already raised?”	`HashSet` unique keys	Duplicate suppression for Part 2 feedback
“Moisture for rack1 between T1–T2?”	Per-device **AVL** tree	Range walk ≈ log n + k; `nodesVisited` proves it
“Who is linked on the radio mesh?”	**Adjacency-list** `MeshGraph`	Natural for neighbour scans, BFS, Prim
“Cheapest backbone to the gateway?”	**Prim MST** parent map	Gateway-rooted hops for `/mesh` route tracking
“Critical alert vs FIFO backlog?”	Priority queue + FIFO queue	Severity (+ time tie-break) vs arrival order

Learning: structures are chosen for the **query the UI/API must answer**, then seeded so the demo is deterministic. Replacing a list with an AVL only paid off because ingest writes both and the range API exposes efficiency metrics operators can see.

7.2 Algorithms — mesh theory applied to a hydroponics gateway

- **BFS vs DFS:** BFS from `sx-gateway` gives hop-distance and healing paths; DFS remains available for deep exploration, but reachability reporting prefers BFS because “how many hops to

a live neighbour?” matters for routing.
- **Online-only walks:** IoT meshes fail partially. Running topology BFS and live BFS side-by-side taught the difference between **down** (offline) and **cut-off** (online but stranded) — both appear in `GET /api/mesh/traversal`.
- **MST as radio budget:** The full mesh (≈25 links) is for resilience; Prim’s tree (≈16 links) is the power-efficient day-to-day backbone. Comparison payloads (~47% cost saved) make the theory tangible for markers.
- **Prim over Kruskal here:** Because telemetry routes **toward a fixed gateway**, growing the tree from the root yields parent hops without a separate rooting pass.

Learning: mesh theory (connectivity, faults, spanning trees) only marks well when tied to **seeded devices** (`sx-pwr-inverter` down, moist sensor path highlight) and operator-facing wording on `/mesh`.

7.3 Async UI and background work — do not block the operator

- **Part 2 feedback** made the cost of sync queue drain obvious: ingest must **enqueue only**; `TelemetryQueueConsumer` (`BackgroundService`) drains priority then FIFO.
- **`PeriodicTimer` on `/commands`** (5s) keeps KPIs and suggestions fresh without a manual reload; it pauses during override/undo so the operator’s action is not raced.
- **`OperatorSessionState`** taught that WASM “pages” are components — filters must live in a scoped/singleton service or they vanish on navigate.
- **`/mesh` loads** topology, traversal, MST, and AVL history asynchronously, with explicit API-down / empty / error callouts so a dead `:5015` is obvious instead of an endless spinner.

Learning: “Full functioning app” means the UI stays usable while the gateway does real work. Async Blazor + a hosted consumer separates **operator latency** from **queue processing**.

7.4 Integration discipline — one store, three pillars

- Sharing **`GatewayStore`** forced freshness, alerts, and mesh online flags to stay consistent (silent sump and mesh-down inverter are the same truth).
- **`SmartX.Shared`** as the contract layer avoided duplicating graph/AVL logic in the browser.
- Docker Compose later reused the same ports/CORS story — containers did not require a second architecture.

Learning: evolving Part 1 → 2 → 3 in one process is harder than three demos, but it matches the brief and makes the GitHub repo tell one story.

7.5 What I would watch in a larger deployment

In-memory state and boot seeds are correct for PROG7312 marking. Production would add durable storage, auth, and horizontal scale — without abandoning the same structures (indexed history, graph snapshot, MST recompute on topology change). Those directions are outlined in Stages **8.7–8.8**.

---

8. Recommended additional technologies

Smart-X already runs locally (and optionally under **Docker Compose**). The following tools would extend it toward a production-style IoT operations platform. Each recommendation is tied to a concrete Part 1–3 gap and justified against **benefits** plus **.NET ecosystem** fit

(security, scalability, performance, workflow).

8.1 Cloud-hosted database (Azure SQL / Cosmos DB / PostgreSQL on Azure)

Replace: in-memory `GatewayStore` lost on every API restart.
Add: persistent storage for sensors, packet history, commands, alerts, and mesh topology snapshots.
Why for Smart-X: markers and demos reset cleanly today; a real hydroponics site needs history across reboots and multi-instance APIs. AVL/range queries would read from indexed time-series or document partitions instead of process RAM.

.NET rationale: EF Core / Dapper / Azure Cosmos SDK for .NET plug into the existing ASP.NET Core DI pipeline without leaving C#. **Security** — Managed Identity + Azure Key Vault (no connection strings in `appsettings`); SQL/Cosmos RBAC instead of shared SA passwords. **Scalability** — multiple Api replicas share one store; Cosmos partition keys (e.g. `sensorId` / site) match fleet growth. **Performance** — indexed range queries replace linear in-memory scans for history windows; keep hot AVL/mesh graphs in memory with durable write-behind. **Workflow** — same `dotnet` + Azure Data Studio / VS tooling already used for the solution.

8.2 GitHub Actions CI/CD

Replace: manual `dotnet build` / "it works on my machine" before push.
Add: workflows on `push`/`pull_request` to restore, build Api+Client, optionally run smoke curls against a test host, and publish Docker images.
Why for Smart-X: the `POE` branch already has 100+ commits; CI would guard mesh/queue regressions when feedback fixes and Part 3 land together.

.NET rationale: Official `actions/setup-dotnet` and `dotnet restore/build/test/publish` are first-class; WASM publish (`Blazor` static) fits the same job matrix as the Api. **Security** — OIDC federation to Azure/GHCR avoids long-lived deploy secrets; Dependabot for NuGet CVEs. **Scalability** — parallel jobs for Api vs Client; cache NuGet packages across runs. **Performance** — fail fast on compile/test before Docker layer builds waste minutes. **Workflow** — PR checks on `POE`/`main` match the existing GitHub remote (`akzz98/SmartX`) with no new IDE required.

8.3 LLM-assisted alert summaries (Azure OpenAI / GitHub Models API)

Replace: operators reading raw HashSet keys and long alert reason strings on `/commands`.
Add: a summarisation endpoint that turns recent priority alerts + disconnect keys into a short operator brief ("irrigation risk on rack 1; inverter mesh relay down — use gateway hop").
Why for Smart-X: Part 2 suggestions already score patterns; an LLM layer would narrate *current* incidents without replacing the deterministic recommendation engine.

.NET rationale: Azure OpenAI / OpenAI .NET clients and Semantic Kernel integrate as scoped services beside existing controllers. **Security** — keep prompts server-side; never send actuator secrets; use Azure AD / API keys in Key Vault; rate-limit the summary endpoint so WASM clients cannot burn quota. **Scalability** — queue summary jobs (reuse PriorityQueue patterns) so ingest latency stays untouched. **Performance** — call LLM only on alert bursts or operator request; cache brief text per disconnect set hash. **Workflow** — Feature flags in `appsettings` toggle summaries without forking the Blazor UI.

8.4 Home Assistant (MQTT / REST integration)

Replace: simulation-only ESP32 traffic into the gateway.
Add: bridge Smart-X commands and telemetry to Home Assistant entities (valves, pumps, moisture sensors) via MQTT or HA REST.
Why for Smart-X: aligns with real SA home/site automation; Part 2 overrides (`OpenValve`, `StartPump`) become physical or HA-virtual actuators while Progressive Disclosure still investigates gateway state.

.NET rationale: `MQTTnet` and `HttpClient` factory are mature .NET packages; a hosted `BackgroundService` can bridge HA ⇔ `GatewayStore` like today's queue consumer. **Security** — long-lived HA tokens in secrets; TLS MQTT; authorize override commands the same way as current API policies before publishing to the bus. **Scalability** — MQTT decouples bursty ESP32 traffic from Api threads; multiple sites = multiple topics/tenants. **Performance** — async publish/subscribe avoids blocking ingest; batch telemetry packs. **Workflow** — local HA + Docker Compose already used for the stack; students can demo without cloud until Azure IoT Hub is added later.

8.5 Deeper containerisation / orchestration (already started)

Current: `docker compose up --build` for Api + nginx WASM client.
Extend: Azure Container Apps or Kubernetes for replicas, health probes, and separate scaling of API vs static client; attach volume/DB for App_Data attachments.
Why for Smart-X: Compose proves packaging; orchestration matters once the cloud DB and CI publish images continuously.

.NET rationale: Existing multi-stage Dockerfiles (`mcr.microsoft.com/dotnet/sdk` / `aspnet`) are the Microsoft-supported path; Container Apps speaks native ASP.NET health endpoints. **Security** — non-root containers, private ACR, managed identity to DB. **Scalability** — scale Api replicas independently of the static nginx Client; KEDA-style scale on queue depth later. **Performance** — layer caching + `dotnet publish` trimmed/ReadyToRun options for cold starts. **Workflow** — same `docker compose` files graduate to ACA/`kubectl` with minimal Dockerfile churn; CI publishes images Actions already built.

8.6 Observability (OpenTelemetry + Application Insights or Grafana)

Replace: console logs and manual curl health checks.
Add: traces for ingest → queue → consumer, metrics for queue depth / MST recompute time, dashboards for disconnect rate.
Why for Smart-X: Part 2 queue behaviour and Part 3 traversal cost become measurable in ops, not only in README curls.

.NET rationale: `OpenTelemetry.Extensions.Hosting` + ASP.NET Core instrumentation is the standard .NET observability stack; Application Insights exporter is one package. **Security** — scrub PII from spans (operator notes, attachment paths); restrict dashboard access. **Scalability** — sampling keeps high-ingest telemetry affordable; metrics (not full traces) for every packet. **Performance** — Activity spans around BFS/MST show when Part 3 algorithms dominate CPU under large meshes. **Workflow** — `dotnet` counters / Aspire dashboard locally, then export to Azure Monitor or Grafana without rewriting controllers.

---

9. Conclusion

Smart-X progressed from telemetry Progressive Disclosure (Part 1), through a real-time command

````
and alert pipeline (Part 2), to a complete network-topology and mesh-routing pillar (Part 3).
The Final PoE artefact is not a separate app: it is the same gateway with trees, graphs, and MST
wired into APIs and the `/mesh` dashboard, with Part 1/2 lecturer feedback addressed and
evidenced in the feedback ledger.

---

## Appendix A — Key source map

| Concern | Location |
| --- | --- |
| Fleet / queues / mesh store | `SmartX.Api/Storage/GatewayStore.cs` |
| Background consumer | `SmartX.Api/Storage/TelemetryQueueConsumer.cs` |
| Mesh seed + down scenario | `SmartX.Api/Simulation/MeshTopologySeeder.cs`,
`MeshDownScenarioSeeder.cs` |
| Mesh APIs | `SmartX.Api/Controllers/MeshController.cs` |
| AVL history | `SmartX.Shared/Collections/AvlTelemetryHistoryTree.cs` |
| BFS / reachability | `SmartX.Shared/Mesh/MeshGraphTraverser.cs`, `MeshReachabilityAnalyzer.cs`
|
| Prim MST | `SmartX.Shared/Mesh/MeshMstBuilder.cs` |
| Topology UI | `SmartX.Client/Pages/Mesh.razor` |
| Feedback ledger | `docs/FEEDBACK-UPDATES.md` |
| Operator how-to | `README.md` |

## Appendix B — Exporting this report to PDF

1. Open `docs/PROJECT-COMPLETION-REPORT.md` in VS Code / Word / a Markdown preview.
2. Print → **Microsoft Print to PDF** (or Export PDF).
3. Save as `docs/PROJECT-COMPLETION-REPORT.pdf` alongside this source.

Alternatively (if Pandoc is installed):

```bash
pandoc docs/PROJECT-COMPLETION-REPORT.md -o docs/PROJECT-COMPLETION-REPORT.pdf
```
````